# Hardness of Coloring with Seven Connected Bundles

Working manuscript · awaiting expert assessment
16 September 2026

## Abstract
We give a deterministic polynomial-time reduction from bipartite list coloring with a global palette of three colors to weighted coloring with seven connected bundles. Six bundles are singletons; the remaining bundle contains the source graph and neutral connectors between its components. Both the weights and the acceptance threshold are fixed constants. A weighted palette forces every accepted target coloring to respect the source lists, so restricting the target coloring recovers a source witness. The construction has linear graph size. The resulting decision problem is NP-complete. The contribution is this restricted endpoint: the palette and precoloring mechanisms themselves are established techniques.

## 1. Introduction
Coloring with bundles charges for the largest color used within each prescribed vertex set. Requiring each set to induce a connected subgraph is a natural structural restriction. Ito et al. [1, Theorem 8] give a polynomial-time algorithm for connected partitions on trees. Their bipartite hardness results with a constant number of bundles [1, Theorems 10–11] do not require those bundles to be connected. This leaves a specific question: can hardness persist with both connected bundles and a constant bundle count?

This paper gives an affirmative answer for seven weighted bundles. The six singleton bundles force a three-color palette by a numerical cost inequality. One connected bundle holds the original vertices and auxiliary vertices that join source components without restricting their color choices. Our main result includes recovery from every accepted target coloring.

**Theorem 1.** Let the target consist of a simple bipartite graph with an ordered partition into seven nonempty connected bundles, the first six singleton. Fix the weights and threshold to

$$(w_1, \dots, w_7) = (4096, 1024, 256, 64, 16, 4, 1), \quad T = 5823. \tag{1}$$

Deciding whether a proper positive-integer coloring has weighted sum of bundle maxima at most $T$ is NP-complete. There is a deterministic polynomial-time Karp reduction from Bipartite List 3-Coloring, with polynomial-time recovery of a source coloring from every accepted target coloring. On an $n$-vertex, $m$-edge source, its output has $O(n + m + 1)$ graph size and $O((n + m + 1)\log(n + 2))$ encoded bits.

The unrestricted number-of-bundles case already includes ordinary sum coloring through singleton bundles. Our result concerns the constant-count restriction. It neither locates the smallest hard connected-bundle count nor classifies the unweighted case. Section 5 compares the construction with older palette and precoloring results.

## 2. Languages and witnesses
A source consists of a simple bipartite graph $G = (V, E)$, with $V = \{0, \dots, n - 1\}$, and explicit lists $L(v) \subseteq \{1, 2, 3\}$. A source witness $\ell$ assigns a color in $L(v)$ to each vertex and different colors to adjacent vertices. Empty lists are permitted. The empty graph is a YES instance. This global-three-color source is NP-complete by Chlebík and Chlebíková [2, Theorem 5], even on simple planar cubic bipartite graphs.

A target witness $\chi$ assigns positive integers to the vertices of a legal target graph $H$, with different colors on each edge. For its ordered bundle partition $(B_1, \dots, B_7)$, define

$$C(\chi) = \sum_{i=1}^{7} w_i \max_{v \in B_i} \chi(v). \tag{2}$$

The witness is accepted exactly when $C(\chi) \le T$. All seven bundles must be nonempty and induce connected subgraphs. Inputs violating the source or target schema are outside the corresponding decision language. The forward map below sends malformed source strings to a legal fixed NO instance.

## 3. A forced palette

Introduce vertices in the order

$$(p_1, \ldots, p_6) = (a_1, b_1, a_2, b_2, a_3, b_3). \tag{3}$$

Place the $a$-vertices on one side of a bipartition and the $b$-vertices on the other. Add exactly the edges $a_i b_j$ with $i \neq j$. Give $p_i$ its own singleton bundle of weight $w_i$. Figure 1 shows all six edges and weights.

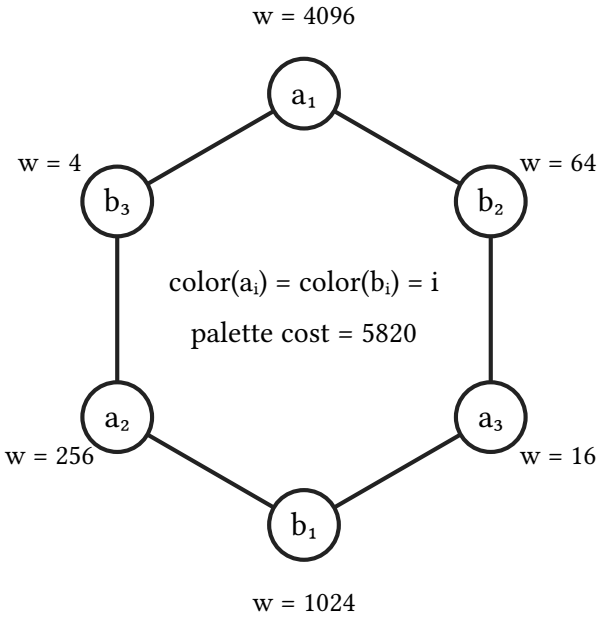



Figure 1: The entire palette graph, a six-cycle with edges $a_i b_j$ for $i \neq j$. Each vertex forms a separate singleton bundle with the indicated weight. The intended coloring assigns color $i$ to both $a_i$ and $b_i$. The two bipartition classes consist of the vertices labeled $a_i$ and $b_i$, respectively.

Let $d = (1, 1, 2, 2, 3, 3)$ be the intended ordered palette coloring. It is proper and its weighted cost is 5820. The weights satisfy

$$w_i > 2 + \sum_{j=i+1}^{6} w_j(d_j - 1) \quad (1 \le i \le 6). \tag{4}$$

The six positive differences between the left and right sides are $(3734, 662, 150, 22, 6, 2)$.

**Lemma 2 (palette forcing).** Suppose $c$ is any proper positive-integer coloring of the palette and $h$ is any positive integer. If $\sum_{i=1}^{6} w_i c_i + h \le 5823$, then $c = d$ and $h \le 3$.

**Proof.** If $c \neq d$, take the earliest differing position $i$. At the first two positions, no positive color is smaller than the intended color. At the next two positions, earlier neighbors exclude color one. At the last two positions, earlier neighbors exclude colors one and two. Since all earlier positions agree with $d$, in every case $c_i \ge d_i + 1$.

All later colors are at least one, as is $h$. Relative to the intended palette cost and a main-bundle contribution of three, the total excess is therefore

$$\sum_{j=1}^{6} w_j c_j + h - 5823 \ge w_i - \sum_{j=i+1}^{6} w_j(d_j - 1) - 2 > 0. \tag{5}$$

This contradicts the budget by Equation 4. Thus $c = d$, and substituting its cost gives $h \leq 3$. □

The lemma assumes neither an optimal coloring nor a prior bound on its colors. In the reduction, $h$ will be the actual maximum of the nonempty seventh bundle.

## 4. The complete reduction rule

### 4.1. Forward construction

Validate the source schema and bipartiteness. For a valid nonempty source, order its components by their smallest vertex. In each component, choose that vertex as representative and orient the component bipartition so that the representative lies on side $A$. Keep every original vertex and edge. Between each consecutive pair of representatives insert a fresh vertex on side $B$ adjacent to both. Put all these vertices in $B_7$. If the source is empty, use one isolated dummy vertex on side $A$ as $B_7$ instead.

Append the palette, with each $a_j$ on side $A$ and $b_j$ on side $B$. For each original vertex $v$ and each color $j \notin L(v)$, join $v$ to the opposite-side palette vertex of color $j$: to $b_j$ if $v$ is on side $A$, and to $a_j$ otherwise. Use the fixed ordered singleton bundles and weights of Theorem 1. Figure 2 explains the two interfaces.

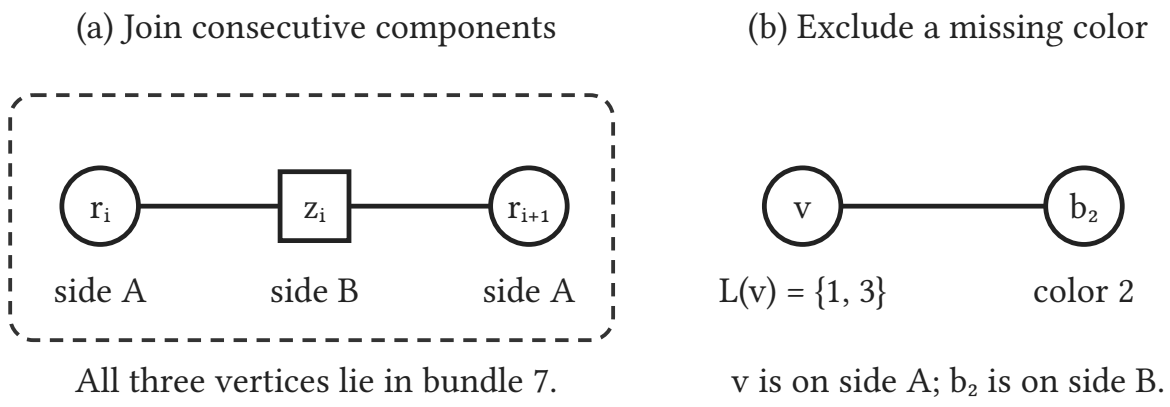



Figure 2: Two local interfaces of the construction. (a) A fresh vertex $z_i$ joins consecutive component representatives $r_i, r_{i+1}$. The dashed boundary denotes their common bundle, not graph edges. For any endpoint colors, $z_i$ has an available color in $\{1, 2, 3\}$. Component interiors are omitted. (b) For a vertex on side $A$ with list $\{1, 3\}$, the edge to $b_2$ excludes color two. Other palette edges are omitted. These are schematic local views.

Assign original vertices the first $n$ indices, then the connectors or dummy, then the palette vertices in their stated order. Sort the output edges. These choices specify a deterministic construction, denoted $f$.

**Lemma 3 (legal output).** Every constructed target is simple and bipartite, and its seven bundles form a connected partition.

**Proof.** Original edges and palette edges are simple and bipartite. Each connector is fresh and joins two distinct side-$A$ representatives. Each list edge joins an original vertex to a distinct opposite-side palette vertex; these edges cannot duplicate any other edge type. Every added edge crosses the declared bipartition. The component chain connects the induced subgraph on $B_7$; in the empty case that subgraph has one vertex. The other bundles are nonempty singletons. They partition the output vertices. □

### 4.2. Witness recovery and equivalence

Given the source and a target coloring, the recovery algorithm reconstructs $f(G, L)$ and checks the coloring's length, positive integer entries, edge constraints and weighted cost. It rejects an invalid recovery input explicitly. For an accepted coloring, it returns the first $n$ colors.

**Lemma 4 (both directions).** A valid source has a list coloring if and only if its constructed target has an accepted coloring. Every accepted target coloring recovers a valid source list coloring.

**Proof.** Given a source coloring, keep its colors on the original vertices. Each connector must avoid at most two colors, so assign it the smallest remaining color in $\{1, 2, 3\}$. Give an empty-source dummy

color one and the palette coloring $d$. Source edges remain proper. Each list edge joins a source vertex to a color absent from its list, so it is proper as well. The maximum on $B_7$ is at most three; the total cost is at most 5823.

Conversely, take any accepted target coloring. Apply Lemma 2 with the actual maximum on $B_7$. The palette is fixed to $d$, and every original vertex has color in $\{1, 2, 3\}$. A list edge forbids exactly its associated missing color. All original edges remain present. Restricting the coloring therefore yields a proper source coloring respecting every list. The empty restriction is valid when $n = 0$. This proves both the NO implication and the recovery claim. □

### 4.3. Totality and resource bounds

For any malformed source string, output the construction for a single vertex with an empty list. This target has seven vertices; its main vertex is adjacent to $b_1, b_2, b_3$. It is legal by Lemma 3 and is NO by Lemma 2, since the main vertex has no available color. Thus malformed strings preserve the language equivalence too.

The implementation reads explicit JSON graphs and lists. It disables Python's decimal-integer digit cap before decoding, then checks the count against the explicit list array before allocating the graph. Invalid JSON and source-schema violations use the fixed NO output. This avoids a runtime-dependent limit on the length of integer tokens. Decimal conversion and validation take polynomial time in the string length.

For a nonempty source with $k$ components, the output vertex and edge counts satisfy

$$|V(H)| = n + k + 5 \leq 2n + 5, \tag{6}$$

$$|E(H)| \leq m + 2(k - 1) + 3n + 6 \leq m + 5n + 4. \tag{7}$$

The empty-source and malformed-source outputs have constant size. Vertex identifiers require $O(\log(n + 2))$ bits, and the numerical parameters are fixed, so the output encoding has $O((n + m + 1) \log(n + 2))$ bits.

Graph validation, component traversal, sorting and serialization all have polynomial worst-case cost in the explicit source length, including the bit cost of reading integer tokens. The constructor performs no coloring search. Recovery reconstructs the same target, checks the supplied witness and returns its prefix; its running time is polynomial in source-plus-witness length.

**Proof of Theorem 1.** Lemmas 3 and 4, the fixed NO output and the resource bounds give the claimed deterministic Karp reduction and recovery algorithm. The source is NP-hard by [2, Theorem 5]. For target membership in NP, every accepted vertex color is at most 5823, since all weights are positive and every vertex belongs to a bundle. A coloring thus has polynomial length, and graph legality, connectivity, properness and cost can be checked in polynomial time. □

## 5. Relation to previous constructions

The mathematical techniques have older precursors. Bodlaender, Jansen and Woeginger [3, Theorem 1] prove hardness of three-color extension on bipartite graphs with a designated precolored vertex for each color. Chlebík and Chlebíková [2, Theorem 6] use precolored neighbors to exclude missing list colors. Murat and Paschos [4, Proposition 1] use the same off-diagonal six-vertex palette, with separated probabilities, for a probabilistic coloring objective. We do not claim that graph or the list encoding as new.

The endpoint here adds fixed weighted maxima over a partition whose seven parts all induce connected subgraphs. The cost inequality forces the palette, and neutral connectors enforce connectivity without changing source satisfiability. The cited precoloring results alone do not impose these two requirements. The bundle constructions in [1, Section 4] permit disconnected parts; the tree algorithm in [1, Section 3.4] has a different graph domain.

This is a modest restriction result. It supports hardness under simultaneous constant bundle count and induced connectivity. Literature inspection through 16 September 2026 did not identify an earlier theorem covering this endpoint. That assessment has bounded coverage: unindexed equivalents remain possible, and the comparison with [4] used its indexed primary construction rather than a successfully retrieved full PDF. Expert assessment of novelty remains needed.

## References


[1] T. Ito, N. Kakimura, N. Kamiyama, Y. Kobayashi, and Y. Okamoto. *Minimum Sum Coloring with Bundles in Trees and Bipartite Graphs.* ISAAC 2025, LIPIcs 359, Article 40. Full version: arXiv:2509.15080v1.

[2] M. Chlebík and J. Chlebíková. *Hard coloring problems in low degree planar bipartite graphs.* Discrete Applied Mathematics 154(14), 1960–1965, 2006. doi:10.1016/j.dam.2006.03.014.

[3] H. L. Bodlaender, K. Jansen, and G. J. Woeginger. *Scheduling with incompatible jobs.* Discrete Applied Mathematics 55(3), 219–232, 1994. doi:10.1016/0166-218X(94)90009-4.

[4] C. Murat and V. Th. Paschos. *The Probabilistic Minimum Coloring Problem.* WG 2003. Primary manuscript, Proposition 1.

## A. Verification and reproducibility

Finite checks test the implementation; they do not replace the general proof. Two separately written testing programs execute the candidate in fresh processes without importing it. The prepared source oracle uses Z3 list constraints. The separate verifier enumerates source assignments and uses exact nested maxima for target cost. Both validate actual target graphs and recovered witnesses.

The prepared suite covers ten cases with nine recoveries. A seeded suite covers 160 instances on at most seven vertices, including 126 YES and 34 NO cases, with 347 recoveries. Its seed is 20260916. The separate verifier covers all pairs of lists from $\varnothing, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,2,3\}$ on two vertices, with and without an edge, plus empty and disconnected pinned cases: 74 decisions and 112 recoveries. These counts are not deduplicated.

The prepared and separate suites passed again after a parser-only repair. Eight additional raw malformed-input cases produce legal NO targets, including oversized integers. The seeded evidence predates this repair and tests the unchanged graph construction. Independent review adds five decisions and all 16 accepted target witnesses of those cases, plus eight parser regressions and one legal forward/recovery check. No remaining defect was identified in that review; this is not a guarantee supplied by finite testing.

Reproduction uses Python 3.14.7, NetworkX, Z3 and Typst as installed for the recorded runs. Python must support `sys.set_int_max_str_digits` (available from 3.11). From the repository root, enter the campaign:

```
cd campaigns/seven-connected-bundle-coloring
python3 work/check.py --self-test
python3 work/check.py --candidate work/algorithm.py
python3 work/verify.py --candidate work/algorithm.py
```

The forward and recovery interfaces each read one JSON value from standard input. These commands give a singleton source pinned to color three and a valid coloring of its constructed target:

```
printf '%s\n' '{"n":1,"edges":[],"lists":[[3]]}' |
  python3 work/algorithm.py
printf '%s\n' '{"source":{"n":1,"edges":[],"lists":[[3]]},"target_solution":
[3,1,1,2,2,3,3]}' |
  python3 work/algorithm.py --extract
```

The recovered source witness is `[3]`. To compile the paper:

```
typst compile work/manuscript.typ work/manuscript.pdf
```

Detailed source and witness predicates are in `work/contract.md`. Raw checks, including the finite seeded suite and parser regressions, are under `rounds/001/`. The independent reports and their own scripts are under `reviews/initial/` and `reviews/parser-and-attribution/`. The scripts declare finite scopes and impose no wall-clock timeout.